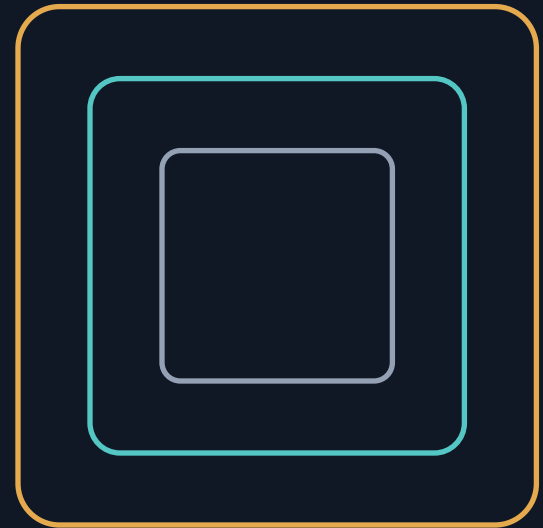


CIS 4020 · DATA STRUCTURES

Linked List

The wall, the node, and the four shapes it can take





What Is an ADT?

- A collection of data, plus the operations allowed on it
- Enables modularity, information hiding, and data abstraction
- A data structure is the construct that stores the collection



The interface is the wall. The program only ever talks to add / remove / find / display; it never touches the data structure directly.



Common ADTs

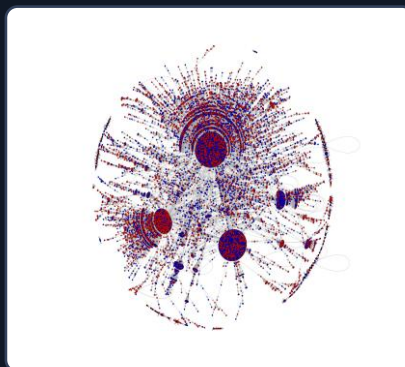
The linked list is one tool. Stacks, queues, tables, trees, and graphs are built the same way: items connected to their neighbors.



Tables



Trees



Graphs



Stacks



Queues



Linked List Terminology

Terminology

Node: holds data and a link to the next node.

Head: first node, no predecessor.

Tail: last node, no successor.

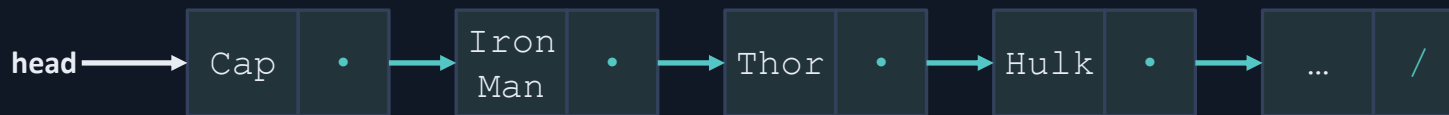
Predecessor / successor: the node before / after a given node.

Basic operations

Create, add, retrieve, remove, destroy.

Is it empty? How many items?

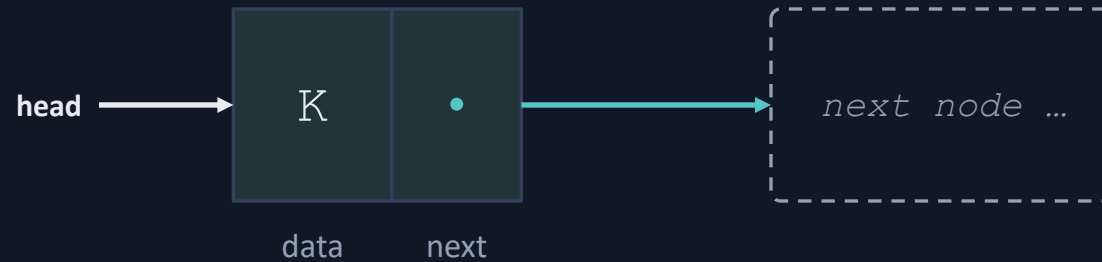
Once these exist, build on them: display, sort, search, find-and-replace.



Example list: an ordered Avengers roster, one node per hero



Anatomy of a Node



data can be anything: a primitive, an array, an object, even another ADT.

next a reference to the following node. The head reference lives outside every node; it just marks where the list begins.



Array-based vs. Pointer-based

Array-based

Sequential: to reach element 4 you pass through 0, 1, 2, 3.
Insert or remove means shifting elements.
You keep track of the last index in use.

Pointer-based (most powerful)

Nodes hold the data and a pointer to the next node.
Java hides the address, but you can still see object identity.
C/C++ lets you see the pointer and address directly.

Data	Data	Data	Data	Data	Data	Data	Data	Data	Data
0	1	2	3	4	5	6	7	8	9

An array-based list: contiguous cells, accessed by index



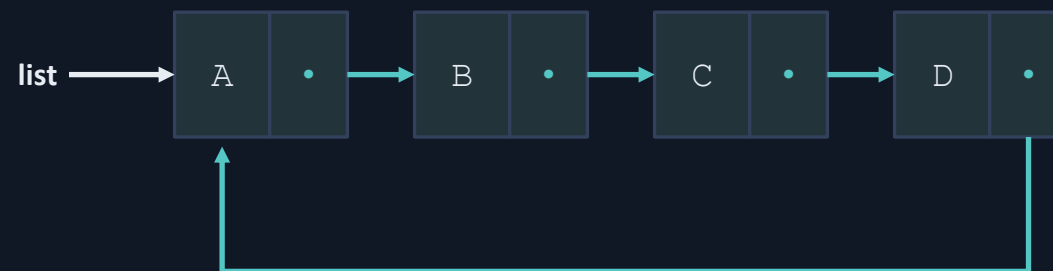
Singly Linked vs. Circular Linked

Singly linked



Walks forward only. The last node's next field is null.

Circular linked



Same idea, but the last node points back to the first. Stop the walk by checking whether you are back where you started, not by checking for null.



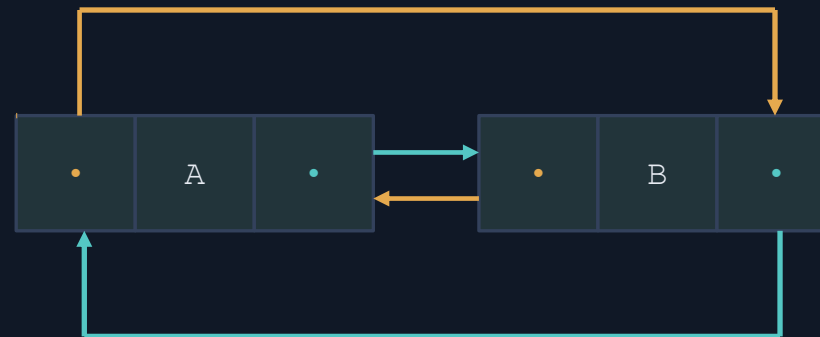
Doubly Linked vs. Circular Doubly Linked

Doubly linked



Two links per node, so you can walk in either direction. The extra pointer costs memory but makes removing a known node cheaper.

Circular doubly linked



Both links wrap around. The most flexible shape; some implementations add a dummy head node so an empty list is not a special case.



A Few Closing Thoughts

- One of the most fundamental data structures you will use all semester
- A stack restricts add and remove to the same end; a queue uses different ends. Both are special-case linked lists
- Trees, graphs, and tables all reuse the same idea: items connected to their neighbors
- Linear data structures are conceptually simple, but implementations can be tricky to get exactly right
- Using them correctly inside a larger algorithm is its own skill, worth the practice